

코흐 눈송이 인터랙티브 시뮬레이션 탐구 활동지

[활동 1] 화면 깜빡임 없는 초고속 렌더링 제어 원리

단계(depth)가 4가 되면 선분이 768개로 늘어납니다. 거북이가 하나씩 그리는 딜레이를 없애고 키보드를 누르는 즉시 화면을 갱신하는 렌더링 메커니즘을 분석해 봅시다.

명령어	역할 및 동작 원리	생략 시 나타나는 문제점
tracer(0)	그리는 중간 애니메이션 과정을 끄고 연산만 메모리에서 수행	선분 768개를 일일이 그리느라 화면이 멈춤
clear()	이전에 그려진 눈송이와 상단 안내 텍스트를 화면에서 삭제	이전 단계의 그림 위에 새 그림이 겹쳐서 그려짐
update()	메모리에 완성된 전체 그림을 화면에 한 번에 출력	화면에 아무것도 나타나지 않음

[활동 2] lambda (람다) 함수 이해하기: 이름 없는 한 줄 함수

파이썬의 onkey (함수, "키")는 실행할 함수의 이름만 넘겨받아야 하며 render (1)처럼 괄호와 인자를 직접 넣을 수 없습니다. 이를 해결하는 **lambda 표현식**을 알아봅시다.

1. 문제 상황: onkey (render (1), "1")이라고 쓰면 키를 누를 때 실행되는 것이 아니라 **프로그램이 켜지자마자 실행**되는 오류가 발생합니다.

2. 일반 def 방식의 한계: 인자를 넘기기 위해 아래처럼 set_0 ()부터 set_4 ()까지 의미 없는 함수를 5개나 따로 정의해야 합니다.

[방식 A] 일반 def 함수 정의 (5번 반복) <pre>def set_0(): render(0) def set_1(): render(1) def set_2(): render(2) ... onkey(set_0, "0") onkey(set_1, "1")</pre>	[방식 B] lambda 익명 함수 사용 (간결함) # lambda 문법: lambda : 실행할_코드 <pre>onkey(lambda: render(0), "0") onkey(lambda: render(1), "1") onkey(lambda: render(2), "2")</pre>
--	---

핵심 요약: lambda: render (1)은 지금 실행하지 말고, 나중에 1번 키가 눌렸을 때 render (1)을 실행해 주는 이름 없는 임시 함수를 그 자리에서 즉석으로 만들어 전달하는 문법입니다.

질문 1. onkey에 render(3)을 직접 쓰면 안 되는 이유는?

작성란:

질문 2. def 대신 lambda를 사용했을 때의 장점은?

작성란:

시뮬레이터 전체 코드 제어 및 인터랙션 탐구

[활동 3] 시뮬레이터 전체 코드 완성하기 (빈칸 채우기)

고속 렌더링 함수와 lambda 키보드 이벤트 연결 코드를 완성해 봅시다.

```
def render(depth):
    tracer(0) # 1. 중간 그리기 과정 생략
    _____() # 2. 이전 단계의 그림과 텍스트를 깨끗이 지움

    draw_header(depth) # 3. 상단에 현재 깊이 안내 텍스트 출력

    # 4. 화면 중앙 정렬 위치로 이동 후 눈송이 그리기
    penup()
    goto(-130, 40)
    setheading(0)
    pendown()
    color("#00fff5")
    width(1.5)

    for _ in range(3):
        koch(current_length, depth)
        right(_____) # 정삼각형 외각 회전

    _____() # 5. 완성된 화면을 한 번에 표시

# 키보드 이벤트 리스너 등록 (lambda 활용)
listen()
onkey(lambda: render(_____), "0")
onkey(lambda: render(1), "1")
onkey(lambda: render(2), "2")
onkey(lambda: render(3), "3")
onkey(lambda: render(4), "4")
```

[활동 4] 시뮬레이터 조작을 통한 단계별 화면 관찰 기록

시뮬레이터를 실행한 후 키보드 0, 1, 2, 3, 4 키를 차례로 누르며 화면의 변화와 총 선분 개수를 기록해 봅시다.

키 입력	깊이 (depth)	화면 형태	총 선분 개수 (정삼각형 3개 변 기준)
'0'	0단계	기본 정삼각형	3개 (3×4^0)
'1'	1단계	육각별 형태	12개 (3×4^1)
'2'	2단계	돌기 돌은 별	_____ 개 (3×4^2)
'3'	3단계	정밀한 눈송이	_____ 개 (3×4^3)
'4'	4단계	극도로 섬세한 프랙탈 눈송이	768개 (3×4^4)

[활동 5] 시뮬레이션 심층 탐구: 컴퓨터 그래픽스와 재귀의 힘

시뮬레이터를 직접 조작해보며 확인한 컴퓨터 그래픽스 기술과 재귀 알고리즘의 특징을 정리해 봅시다.

질문 1. tracer(0)가 없다면 4단계에서 어떤 현상이 일어날까요?

작성란:

질문 2. 코드가 짧음에도 768개의 선분이 그려지는 재귀의 원리는?

작성란: